

Final Report: Stability and Correctness Testing of Python's `marshal` Module

Repository: ``https://github.com/<your-username>/marshal-stability-tests``

Local validation environment: CPython 3.13.5, Linux x86_64, ``marshal.version == 4``

AI assistance statement: ChatGPT was used to help structure the test plan, draft the report, and review code clarity. The tests were run and checked manually before submission.

1. Objective and Background

The objective of this assignment is to develop a test suite that checks whether Python's `marshal` module is stable and correct. In this report, **stable** means that the same input object produces hash-identical serialized bytes under repeated execution.

Correct means that supported values can be serialized and deserialized without losing the relevant value semantics.

The Python documentation describes `marshal` as an internal Python object serialization module mainly used for `.pyc` files. It is architecture-independent, but it is not a general persistence format and may change between Python versions. The documentation also warns that `marshal` is not secure for untrusted input. This test suite therefore treats cross-version byte equality as a risk to investigate, not as a guaranteed requirement.

2. Test Suite Overview

The test suite is implemented with the standard-library `unittest` framework, with no third-party dependencies. The repository is organized as follows:

- ``marshal_stability/cases.py``: representative test inputs selected with black-box testing techniques.
- ``marshal_stability/fingerprint.py``: helpers for SHA-256 fingerprints and round-trip comparison.
- ``tests/test_marshal_stability.py``: the main unit test suite.
- ``scripts/collect_fingerprints.py``: a script that records hash fingerprints and runtime metadata.
- ``github/workflows/tests.yml``: a GitHub Actions workflow for OS and Python-version matrix testing.

The local command used for validation was:

```
python -m unittest discover -s tests -v
```

Result: **11 tests passed**. The fingerprint script recorded **40 representative values**, and all were stable within the local run.

3. Testing Strategies Applied

3.1 Equivalence Partitioning

The input space was divided into supported equivalence classes based on the documented `marshal` types: numeric values, strings, bytes-like values, containers, singletons, and code objects. For each class, the suite checks repeated serialization stability and round-trip correctness.

Examples:

- numeric values: ``int``, ``bool``, ``float``, ``complex``;
- text and binary values: ``str``, ``bytes``, ``bytearray``;
- containers: ``tuple``, ``list``, ``dict``, ``set``, ``frozenset``;
- singletons: ``None``, ``Ellipsis``, ``StopIteration``.

3.2 Boundary Value Analysis

Boundary value analysis was used where the marshal format may plausibly have internal encoding boundaries. The suite includes empty values, one-character strings, 255-character strings, 256-character strings, empty collections, large lists, 32-bit integer boundaries, 64-bit integer boundaries, and big integers.

This technique was chosen because serialization formats often use different encodings for small and large values. Testing only ordinary values such as ``1``, ``"abc"`, and `[1, 2, 3]` would not be sufficient.`

3.3 Floating-Point Special-Value Testing

Floating-point values were treated as a separate risk area. The suite includes ``0.0``, ``-0.0``, ``Inf``, ``-Inf``, and ``NaN``. ``NaN`` requires special comparison because ``NaN != NaN`` by definition. The helper function therefore checks that deserialized NaN values remain NaN instead of using direct equality.

This technique was chosen because floating-point serialization can be sensitive to bit-level representation, platform behavior, and special values.

3.4 Recursive and Cyclic Structure Testing

The suite creates a list that contains itself and verifies that:

1. repeated ``marshal.dumps()`` calls produce identical bytes; and`
2. after ``marshal.loads()`, the loaded list still refers to itself.`

This is important because recursive containers exercise object-reference handling in the marshal format.

3.5 Deterministic Fuzzing

The suite uses a fixed random seed to generate 100 nested supported values. This gives broader coverage than manually selected examples while keeping the test deterministic and repeatable.

Random fuzzing without a seed was intentionally avoided because it can make failures difficult to reproduce. Deterministic fuzzing is more appropriate for an assignment and for continuous integration.

3.6 Cross-Process Hash-Seed Testing

The suite starts subprocesses with different ``PYTHONHASHSEED`` values and compares fingerprints for dictionaries, sets, frozensets, and tuples. This targets a potential source of non-determinism: hash-dependent container ordering.

The local result showed no hash differences for the selected values across ``PYTHONHASHSEED=1..5``.

3.7 Light White-Box Testing

A limited white-box approach was used by considering documented feature gates and behavior: recursive containers, format versions, and the ``allow_code`` parameter. However, I did not attempt complete all-definitions or all-uses coverage of CPython's C implementation. That would require instrumentation of the CPython interpreter itself and is beyond the scope of this assignment.

4. Traceability Matrix

Requirement or risk	Test technique	Test(s)	Result
Same input should produce identical bytes within one process	Equivalence partitioning	<code>`test_repeated_dumps_are_hash_id` entical`</code>	Passed
<code>`dump()`` and <code>`dumps()`` should produce equivalent bytes</code></code>	API consistency testing	<code>`test_dump_and_dumps_produce_s` ame_bytes`</code>	Passed
Supported values should deserialize correctly	Equivalence partitioning	<code>`test_round_trip_supported_values`</code>	Passed
Empty and large values should behave correctly	Boundary value analysis	Covered by <code>`stable_cases()``</code>	Passed
32-bit, 64-bit, and big integers should serialize stably	Boundary value analysis	Covered by <code>`stable_cases()``</code>	Passed
255/256-character strings should serialize stably	Boundary value analysis	Covered by <code>`stable_cases()``</code>	Passed
<code>`NaN`, <code>`Inf`, <code>`-Inf`, and <code>`-0.0` should not cause instability</code></code></code></code>	Special-value testing	<code>`test_special_float_values_remain_s` table`</code>	Passed
Recursive containers should remain stable and correct	Edge-case testing	<code>`test_recursive_list_round_trip_pres` erves_cycle`</code>	Passed
Random nested supported values should be stable	Deterministic fuzzing	<code>`test_deterministic_fuzz_values_are` _stable_and_correct`</code>	Passed

Hash randomization should not affect selected containers	Cross-process testing	<code>`test_cross_process_hash_seed_stability`</code>	Passed
Unsupported objects should fail predictably	Negative testing	<code>`test_unsupported_objects_raise_value_error`</code>	Passed
Invalid marshal streams should fail predictably	Negative testing	<code>`test_invalid_input_raises_documented_exception`</code>	Passed
Extra bytes after one valid object should follow documented behavior	Specification testing	<code>`test_loads_ignores_trailing_bytes`</code>	Passed
Code objects should be blockable when the runtime supports <code>`allow_code`</code>	White-box/specification testing	<code>`test_code_objects_can_be_blocked_when_supported`</code>	Passed

5. Findings

The local test run found **no non-determinism** for the selected supported inputs on CPython 3.13.5/Linux. Repeated serialization produced hash-identical bytes for all 40 representative cases collected by ``scripts/collect_fingerprints.py``.

The test suite also confirmed several correctness behaviors:

- ``marshal.dump()`` to a ``BytesIO`` object produced the same bytes as ``marshal.dumps()``.
- Supported values round-tripped successfully.
- ``bytearray`` deserialized as ``bytes``, which matches the documented behavior that bytes-like objects are marshalled as bytes.
- A recursive list remained recursive after deserialization.
- ``loads()`` ignored trailing bytes after a valid marshal object.
- Unsupported objects such as ``object()`` and a lambda raised ``ValueError`` when passed to ``dumps()``.
- Code objects were rejected when ``allow_code=False`` on the local Python version that supports this parameter.

The most important interpretation is that the module appeared stable **within the same Python version and tested runtime conditions**. This does not mean the byte format is stable across Python versions. In fact, the official documentation explicitly allows marshal format changes between Python versions.

6. Limitations and Shortcomings

The suite cannot prove complete stability for all possible inputs. It samples representative cases and fuzzed values, but the number of Python objects is effectively unbounded.

The local run was performed on one operating system and one Python version. The included GitHub Actions workflow is designed to improve this by running on Linux, Windows, and macOS across multiple Python versions, but those CI results must be checked after the repository is uploaded.

The suite does not test malicious byte streams beyond a small invalid-input example. This is intentional because the documentation states that ``marshal`` should not be used with untrusted input.

The suite also avoids treating cross-version byte equality as a required pass condition. Since the marshal format may change between Python versions, cross-version byte differences should be reported as compatibility findings rather than automatically classified as bugs.

Finally, code objects are tested only for the ``allow_code=False`` behavior when the runtime supports it. Full code-object compatibility is intentionally excluded because the documentation states that code-object formats are not compatible between Python versions.

7. Conclusion

The test suite provides a focused and reproducible way to evaluate ``marshal`` stability and correctness. It applies equivalence partitioning, boundary value analysis, special-value testing, recursive-structure testing, deterministic fuzzing, negative testing, and limited white-box/specification-based testing.

The local results did not reveal a stability bug for the tested cases. The main practical finding is that ``marshal`` is stable for representative values within the same CPython runtime, but it should not be treated as a long-term, secure, or cross-version persistence format.

8. References

- Python Software Foundation. ``marshal`` - Internal Python object serialization. <https://docs.python.org/3/library/marshal.html>
- Python Software Foundation. PEP 8 - Style Guide for Python Code. <https://peps.python.org/pep-0008/>
- CPython source repository. <https://github.com/python/cpython>